

Hands-On Exercise 2: Web API using .NET Core with Swagger

Scenario

The objective of this exercise is to enhance the existing ASP.NET Core Web API application by integrating Swagger for API documentation and testing. The application is also tested using Postman, and the controller route is modified to demonstrate custom routing.

Objective

- Install and configure Swagger in an ASP.NET Core Web API.
- Generate API documentation using Swagger UI.
- Test Web API methods using Swagger and Postman.
- Understand the usage of Route attributes in Web API.
- Modify the controller route and verify the updated endpoint.

Project Name

FirstWebAPI

Implementation

Step 1: Open Existing Project

Opened the existing **FirstWebAPI** project created in Lab 01.

Step 2: Install Swagger Package

Installed the Swagger package using NuGet Package Manager.

Install-Package Swashbuckle.AspNetCore -Version 6.6.2

Step 3: Configure Swagger

Configured Swagger in **Program.cs**.

```
builder.Services.AddEndpointsApiExplorer();
```

```
builder.Services.AddSwaggerGen();
```

Enabled Swagger middleware.

```
app.UseSwagger();
```

```
app.UseSwaggerUI();
```

Step 4: Create EmployeeController

Created a new controller named **EmployeeController** inside the **Controllers** folder.

```
using Microsoft.AspNetCore.Mvc;
```

```
namespace FirstWebAPI.Controllers  
{
```

```

[Route("api/[controller]")]
[ApiController]
public class EmployeeController : ControllerBase
{
    [HttpGet]
    [ProducesResponseType(StatusCodes.Status200OK)]
    public IActionResult GetEmployees()
    {
        var employees = new List<string>
        {
            "John",
            "David",
            "Smith"
        };

        return Ok(employees);
    }
}

```

Step 5: Build the Project

Built the project using:

Build → Rebuild Solution

Step 6: Run the Application

Executed the application using:

Ctrl + F5

Swagger UI opened automatically.

URL:

<https://localhost:7086/swagger>

Step 7: Test GET Method Using Swagger

Selected **GET /api/Employee**, clicked **Try it out**, and then clicked **Execute**.

Response:

```

[
  "John",
  "David",
  "Smith"
]

```

Status Code:

200 OK

Step 8: Test API Using Postman

Request Type:

GET

Request URL:

https://localhost:7086/api/Employee

Response:

```
[  
  "John",  
  "David",  
  "Smith"  
]
```

Status:

200 OK

Step 9: Modify Route Attribute

Changed the route attribute in **EmployeeController**.

Original:

```
[Route("api/[controller]")]
```

Modified:

```
[Route("api/Emp")]
```

Executed the application again.

New URL:

https://localhost:7086/api/Emp

The API returned the employee list successfully.

Result

The existing ASP.NET Core Web API application was successfully configured with Swagger using the Swashbuckle.AspNetCore package. The API endpoints were tested using both Swagger UI and Postman. The controller route was modified from **Employee** to **Emp**, and the updated endpoint was accessed successfully, demonstrating custom routing in ASP.NET Core Web API.

AI/M xFeed xInbo xCog xEF C xSwag xSwag xSwag xSwag xSwag xlocal xSwag xAsk Gemini

localhost:7086/swagger/index.html

GmailrutwaveMapsAll Bookmarks

ExecuteClear

Responses

Curl

curl -X 'GET' \n'https://localhost:7086/api/Employee' \n-H 'accept: */*'

Request URL

https://localhost:7086/api/Employee

Server response

Code	Details
200	Response body ["John", "David", "Smith"] Response headers content-type: application/json; charset=utf-8 date: Wed, 01 Jul 2020 15:56:12 GMT server: Kestrel

Responses

Code	Description	Links
200	OK	No links